

Types for Programs and Proofs

What exactly are the rules of type theory?

Thierry Coquand

First part: back to lecture 5

We ran out of time on the *deduction theorem*: we finish it, and then ask the question it leaves open. The file is `peirce.agda`

- the Hilbert system *K*, *S*, *modus ponens*: the deduction theorem
- derivations from *no* hypothesis: the family `Deriv`
- how to show that a formula is *not* derivable: a *model*, and *soundness* by induction on the derivation
- the truth tables — no atom is derivable; then Peirce's law, and a model with *two stages of knowledge*

The system, once more

```
data _⊢_ : context -> Form -> Set where
  var : {X : Form} {G : context} -> isIn G X -> G ⊢ X
  axK : {X Y : Form} {G : context} -> G ⊢ X => (Y => X)
  axS : {X Y Z : Form} {G : context} ->
    G ⊢ (X => (Y => Z)) => ((X => Y) => (X => Z))
  mp  : {Y X : Form} {G : context} -> G ⊢ Y => X -> G ⊢ Y -> G ⊢ X
```

Two axiom schemes and one rule; an element of $G \vdash X$ *is* a derivation tree

Deduction theorem: if $\Gamma, X \vdash Y$ then $\Gamma \vdash X \rightarrow Y$

No rule *introduces* an implication; the theorem says we may argue as if one did

The proof, and where S and K come from

```
dedthm : {G : context} {X Y : Form} -> G , X ⊢ Y -> G ⊢ X => Y
dedthm (var x)    = dedlem3 x
dedthm axK        = dedlem2 axK
dedthm axS        = dedlem2 axS
dedthm (mp h h1) = mp (mp axS (dedthm h)) (dedthm h1)

dedlem1 : {G : context} {X : Form} -> G ⊢ X => X
dedlem1 {G} {X} = mp (mp axS axK) (axK {Y = X => X})
```

Four clauses, one per constructor: this *is* the induction principle of the family, used by pattern matching. A hypothesis we do not use needs K ; the hypothesis itself needs $I = SKK$; and the case of `mp` is *exactly* what S says — the two axioms are what the induction needs

Derivations from no hypothesis

```
data Deriv : Form -> Set where
  K  : {X Y : Form} -> Deriv (X => (Y => X))
  S  : {X Y Z : Form} -> Deriv ((X => (Y => Z)) => ((X => Y) => (X => Z)))
  MP : {Y X : Form} -> Deriv (Y => X) -> Deriv Y -> Deriv X

derivId : (X : Form) -> Deriv (X => X)
derivId X = MP (MP S (K {Y = X => X})) (K {Y = X})
```

The same rules with `var` *removed*: there is then no context left, and the family is indexed by a formula only. This is the system of Hilbert

Removing one constructor from an inductive family is a *small* change to write and a large one to reason about: now the family has *three* constructors, so an induction has three cases

Deriv X and nil $\vdash$ X

```
derivToHil : {G : context} {X : Form} -> Deriv X -> G  $\vdash$  X
derivToHil K          = axK
derivToHil S          = axS
derivToHil (MP d d1) = mp (derivToHil d) (derivToHil d1)
```

```
hilToDeriv : {X : Form} -> nil  $\vdash$  X -> Deriv X
hilToDeriv (var ())
hilToDeriv axK      = K
hilToDeriv axS      = S
hilToDeriv (mp d d1) = MP (hilToDeriv d) (hilToDeriv d1)
```

Left to right: a derivation without hypotheses can be read in *any* context, so this direction is also weakening

Right to left: the case `var` *cannot occur*, since `isIn nil X` is empty — no position in the empty list. Agda sees this and the case is written `()`

More than an equivalence

```
roundDeriv : {X : Form} -> (d : Deriv X) ->
    Id (Deriv X) (hilToDeriv (derivToHil d)) d
roundDeriv K      = refl K
roundDeriv S      = refl S
roundDeriv (MP d d1) = cong2 MP (roundDeriv d) (roundDeriv d1)
```

and symmetrically `roundHil`. The two translations are *inverse to each other*: not only is one formula derivable when the other is, the *derivation trees* correspond one to one

Both proofs are an induction on the derivation, with `cong2` of lecture 5 in the case of the rule

This is the kind of statement one would not bother making on paper, and which costs three lines here

How to show that a formula is *not* derivable?

Everything we did so far was *building* derivations. Now we want

$\text{not } (\text{Deriv } X)$ that is $\text{Deriv } X \rightarrow \perp$

Searching and failing proves nothing: the statement is about *all* finite trees

So we build a *model*: an interpretation of the formulae which *validates* K and S and is *preserved* by *modus ponens*. *Soundness*, by induction on the derivation — one case per rule, exactly as for `dedthm` — is then a function turning a derivation of X into a proof that X holds in the model, and if X fails there we have our element of $\perp$ — the pattern of “zero is not one” in lecture 5

The simplest model: the truth tables

```
impB : Bool -> Bool -> Bool          -- the truth table of =>
impB false y = true
impB true  y = y

True : Bool -> Set                    -- a Boolean read as a proposition
True false = ⊥
True true  = ⊤

evalB : (Nat -> Bool) -> Form -> Bool
evalB v (atom n)      = v n
evalB v (imply X Y) = impB (evalB v X) (evalB v Y)
```

An interpretation is a function $v : \text{Nat} \rightarrow \text{Bool}$, one value per atom, and `evalB v` extends it to all formulae *by recursion on the formula*

`True` turns a truth *value* into a *type* one can *prove*

Soundness, for the truth tables

```
lemK : (a b : Bool) -> True (impB a (impB b a))
lemK false b      = tt
lemK true  false = tt
lemK true  true  = tt

soundB : (v : Nat -> Bool) -> {X : Form} -> Deriv X -> True (evalB v X)
soundB v (K {X} {Y})      = lemK (evalB v X) (evalB v Y)
soundB v (S {X} {Y} {Z}) = lemS (evalB v X) (evalB v Y) (evalB v Z)
soundB v (MP {Y} {X} d d1) =
  lemMP (evalB v Y) (evalB v X) (soundB v d) (soundB v d1)
```

The two axioms are *tautologies*, and `lemMP` says that the table of implication is what it should be: all three are checked by trying the finitely many truth values, so the proof is by *computation*. Then one case per constructor of `Deriv`

No atom is derivable

```
vFalse : Nat -> Bool  
vFalse n = false
```

```
notAtom : (n : Nat) -> not (Deriv (atom n))  
notAtom n d = soundB vFalse d
```

Interpret *every* atom by false. Then `evalB vFalse (atom n)` is false, so `True (evalB vFalse (atom n))` is the empty type — and `soundB vFalse d` is an element of it

Read the term: we *apply* soundness to the derivation. The whole argument is one function application, because all the work is in `soundB`

So the system does not prove everything. What it proves is *contained* in the tautologies — but does it prove them all?

Peirce's law

$$((A \rightarrow B) \rightarrow A) \rightarrow A$$

A classical tautology — and this is again checked by computation

```
lemPeirce : (a b : Bool) -> True (impB (impB (impB a b) a) a)
lemPeirce false b      = tt
lemPeirce true  false = tt
lemPeirce true  true  = tt
```

So *no* Boolean interpretation can refute it, whatever values we give the atoms — yet it is not derivable from *K*, *S* and *modus ponens*. We need a finer notion of model

Two stages of knowledge

Interpret a formula not by a truth value, but by what is *known* at each stage of a growing knowledge. Two stages will do, $w_0 \leq w_1$, to be read “ w_1 is a *later stage* than w_0 ”

```
data World : Set where
  w0 w1 : World
```

```
_≤_ : World -> World -> Set          -- defined by recursion, so it computes
w0 ≤ v  = ⊤
w1 ≤ w0 = ⊥
w1 ≤ w1 = ⊤
```

As for $\leq$ on Nat in lecture 5, defining the order by *recursion* makes $w0 \leq w1$ be $\top$: tt is a proof, and leRef1, leTrans are proved by cases

What an atom may be

Knowledge only grows: an atom known at w_0 is known at w_1 . There are therefore exactly *three* possibilities, and we list them

```
data Val : Set where
  never half always : Val

holds : Val -> World -> Set
holds never w = ⊥
holds half w0 = ⊥           -- unknown at w0, known at w1
holds half w1 = ⊤
holds always w = ⊤
```

The fourth combination — true at w_0 , false at w_1 — is the one which is *not* monotone; that the three others are is `holdsMono`

An interpretation of the atoms is a function `Nat -> Val`

Forcing

```
force : (Nat -> Val) -> World -> Form -> Set
force V w (atom n)      = holds (V n) w
force V w (imply X Y) = (u : World) -> w ≤ u ->
                        force V u X -> force V u Y

mono : (V : Nat -> Val) -> (X : Form) -> (u v : World) ->
      u ≤ v -> force V u X -> force V v X
mono V (atom n)      u v p h = holdsMono (V n) u v p h
mono V (imply X Y) u v p h = \ z q hx -> h z (leTrans u v z p q) hx
```

Again by recursion on the formula — but the value is now a *type* indexed by the world, not a Boolean

Everything is in the clause for $\rightarrow$: $X \rightarrow Y$ is known at w when at *every later stage* u , knowing X gives Y . It is a statement about the *future*. Monotonicity for an implication is then just transitivity

Soundness, for forcing

```
sound : (V : Nat -> Val) -> {G : context} -> {X : Form} ->
        G ⊢ X -> (w : World) -> forceC V w G -> force V w X

sound V (var i) w g = lookupC V i w g
sound V (axK {X} {Y}) w g = \ u p hx v q hy -> mono V X u v q hx
sound V (axS {X} {Y} {Z}) w g =
  \ u p f v q h z r hx ->
    f z (leTrans u v z q r) hx z (leRef1 z) (h z r hx)
sound V (mp d d1) w g = sound V d w g w (leRef1 w) (sound V d1 w g)
```

A context is forced when all its formulae are (`forceC`, a conjunction), and `lookupC` reads a hypothesis out of it

K is where *monotonicity* is used: the proof of *X* received at stage *u* has to be kept until the later stage *v*. *S* is bookkeeping of three stages $u \leq v \leq z$. *Modus ponens* uses the premise at the *present* stage, whence `leRef1`

Peirce's law is not derivable

Take $A = \text{half}$ (unknown at w_0 , known at w_1) and $B = \text{never}$

Then $A \rightarrow B$ is forced *nowhere*: at w_0 it fails because of the later stage w_1 , where A is known and B is not; at w_1 it fails at w_1

```
V1 : Nat -> Val                                -- V1 zero = half, V1 (suc n) = never

premise : force V1 w0 ((A => B) => A)
premise w0 p f = efq (f w1 tt tt)
premise w1 p f = tt

notPeirce : not (Deriv peirce)
notPeirce d = sound V1 (derivToHil d) w0 tt w0 tt premise
```

So $(A \rightarrow B) \rightarrow A$ is forced at w_0 *vacuously*, while A is not — and soundness turns any derivation into an element of $\perp$

What the model says

- the *same* interpretation refutes other classical laws, for instance $((A \rightarrow B) \rightarrow B) \rightarrow ((B \rightarrow A) \rightarrow A)$ (notPeirce2)
- the Boolean model is the *special case* where the atoms take the values `never` and `always` only: forcing at w_0 and at w_1 then agree, the quantification over later stages is vacuous, and we are back to the truth tables. The value `half` is the whole point
- the model is *finite* and forcing decidable: a decision procedure for this fragment
- Peirce's law added as an axiom scheme gives *classical* implicational logic, and `dedthm` goes through unchanged — it only inspects `var`, `axK`, `axS` and `mp`

First part: what to remember

- to show something derivable, exhibit a derivation; to show it is *not*, give a model and prove *soundness* by induction on derivations
- soundness is a *function*, one clause per rule: a program over the indexed inductive family of lecture 5
- a coarse model refutes little, a finer one refutes more
- the derivable formulae of this fragment are not the classical tautologies: Peirce's law is missing

And now: which rules *exactly* are we allowed to write down?

Exercises, first part

1. Derive $\Gamma \vdash (X \rightarrow Y) \rightarrow (Y \rightarrow Z) \rightarrow (X \rightarrow Z)$ with `dedthm`, and by hand
2. Show with the Boolean model that $((A \rightarrow B) \rightarrow B) \rightarrow A$ is not derivable
3. Show with the two stages $w_0 \leq w_1$ that $\neg\neg A \rightarrow A$ is not derivable, negation being $A \rightarrow \perp$ for a new atom $\perp$
4. Add a third stage w_2 to `peirce.agda`: which of `leRefl`, `leTrans`, `mono`, `sound` have to be changed?

S. Kripke. *Semantical analysis of intuitionistic logic I*, 1965; A. Troelstra, D. van Dalen. *Constructivism in Mathematics*, 1988

The question

Several of you asked, after the last lectures: *what exactly are the rules of type theory?*

So far we have been *using* type theory — writing Agda, giving formation/introduction/elimination/computation rules for one type former at a time

Today we write down a complete system: a *finite list of rules*, with nothing left implicit, so that the question “is this a rule?” has an answer

Then we *represent* that list in Agda, in `typetype.agda`, where a rule is a *constructor* and a derivation is an *element*

Plan of the second part

- what a system of rules consists of; the raw syntax
- the three judgments $\vdash \Gamma$, $\Gamma \vdash M : A$, $\Gamma \vdash M = N : A$
- the *typing* rules: six; the *conversion* rules: nine
- what is *not* in the list, and why the premises are so many
- *Type : Type*: what goes wrong, what survives

The Agda representation: `typetype.agda`

What a system of rules consists of

Three things, and nothing else

- a *raw syntax*: what strings of symbols we are allowed to write at all
- a list of *judgment forms*: the shapes of the assertions we make
- a list of *inference rules*: premises above the line, conclusion below

A *derivation* is a finite tree built from the rules, and a judgment *holds* when it has a derivation — there is no other way

This is what makes induction on derivations possible

The theory we write down

The smallest interesting dependent type theory

- *one* type former: the dependent product $\Pi(x:A)B$
- *one* universe U , with the rule $U : U$

No Σ , no **Nat**, no **Id**, no hierarchy $U_0 : U_1 : \dots$

Every phenomenon of dependent types is already there — types depending on terms, substitution in types, conversion — and the list of rules is short

The price: as a *logic* it is inconsistent (last slides)

The raw syntax

Expressions, given by a grammar

$$M, N, A, B ::= x \mid U \mid \Pi(x:A)B \mid \lambda(x:A)M \mid MN$$

- *no distinction* between terms and types: “ A is a type” will be the judgment $A : U$
- λ carries its *domain*: $\lambda(x:A)M$, not $\lambda x.M$
- a raw expression may be nonsense: $\Pi(x:x)x$ is one

Variables and substitution

$\Pi(x:A)B$ and $\lambda(x:A)M$ *bind* x in B , in M — not in A

$B[a/x]$ is the result of replacing x by a in B , renaming bound variables as needed. Read the bracket as a fraction, a over x : then $x[a/x]$ is a , as it should be

In the implementation there are no names: an expression with at most n free variables is an element of **Expr** n , a variable is an element of **Fin** n (de Bruijn), and $B[a/x]$ is a function **subst1** : **Expr** $(n + 1) \rightarrow \text{Expr } n \rightarrow \text{Expr } n$ defined by structural recursion

Nothing conceptual — but it is the part one has to get right before anything else can be stated

The three judgment forms

$\vdash \Gamma$

Γ is a well formed context

$\Gamma \vdash M : A$

M has type A in Γ

$\Gamma \vdash M = N : A$

M and N are convertible at type A

A context is a list $x_1:A_1, \dots, x_n:A_n$, each A_{i+1} possibly depending on $x_1, \dots, x_i$

The three are defined *simultaneously*: conversion appears among the premises of typing, and typing among the premises of conversion

Two judgments we do *not* have

There is no judgment $\Gamma \vdash A \text{ type}$

being a type is being of type U

There is no judgment $\Gamma \vdash A = B \text{ type}$ either: type equality is *conversion at the type U* , $\Gamma \vdash A = B : U$

This is the economy bought by $U : U$. In a theory with a hierarchy one needs either a separate judgment for types, or an index U_i everywhere

Andreas's presentation had $\Gamma \vdash A \text{ type}$ as a separate form; both are standard, and they say the same thing here

Rules for contexts

$$\frac{}{\vdash ()} \qquad \frac{\Gamma \vdash A : U}{\vdash \Gamma, x:A}$$

The empty context is well formed; and one may extend a context by a *type*, that is, by an expression of type *U in that context*

Note the dependency: *A* is typed in *Γ* , which is the context being built. This is why the judgments are mutually defined

Typing: variables, and the universe

$$\frac{\vdash \Gamma \quad (x:A) \in \Gamma}{\Gamma \vdash x : A}$$

$$\frac{\vdash \Gamma}{\Gamma \vdash U : U}$$

For the variable rule, $(x:A) \in \Gamma$ is not a judgment: it is a *lookup* in the list Γ , a function. In the implementation `lookup  $\Gamma$   $i$` computes the type of the i -th variable (weakened, so that it lives in the whole context)

The second rule is *the* rule of this theory: U is a type, and it is a type *in the sense of U itself*

Typing: the dependent product

$$\frac{\Gamma \vdash A : U \quad \Gamma, x:A \vdash B : U}{\Gamma \vdash \Pi(x:A)B : U}$$

Formation. Note that the conclusion is again of type U : the universe is *closed* under Π

The second premise is typed in the *extended* context, so B may genuinely depend on x

We write $A \rightarrow B$ for $\Pi(x:A)B$ when x does not occur in B

Typing: abstraction and application

$$\frac{\Gamma \vdash A : U \quad \Gamma, x:A \vdash B : U \quad \Gamma, x:A \vdash M : B}{\Gamma \vdash \lambda(x:A)M : \Pi(x:A)B}$$

$$\frac{\Gamma \vdash A : U \quad \Gamma, x:A \vdash B : U \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[a/x]}$$

Introduction and elimination. The *whole* difference with simple types is in the conclusion of the second rule: the type of $f a$ is obtained by *substitution*

This is where a type theory needs the substitution machinery of the previous slide, and where all the difficulties come from

Typing: the conversion rule

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash A = B : U \quad \Gamma \vdash B : U}{\Gamma \vdash M : B}$$

The rule that ties typing to computation: a term keeps its type along *convertible* types

Without it, $f\ a$ could not be used where $B[a/x]$ has been computed: typing would not be stable under evaluation

It is the only rule which is not *syntax-directed*: looking at a term does not tell you whether it was used. This is what makes typechecking a theorem rather than an obvious algorithm

The typing rules, at a glance

wf-empty, wf-extend	the empty context, and $\Gamma, x:A$
ty-var	a variable of the context
ty-U	$U : U$
ty-Pi	formation: $\Pi(x:A)B : U$
ty-Lam	introduction: $\lambda(x:A)M : \Pi(x:A)B$
ty-App	elimination: $f a : B[a/x]$
ty-conv	change of type along $\Gamma \vdash A = B : U$

The premises are on the previous slides; the names are those of `typetype.agda`

Conversion is a judgment

$\Gamma \vdash M = N : A$ is *not* a relation on raw terms: it is a judgment, like the others, with its own rules

Two consequences

- it is *typed*: one does not ask whether M and N are equal, but whether they are equal *at a type* A in a context Γ
- it relates only *well typed* terms — this is visible in the first rule below, whose premise is a typing

This is *judgmental* equality, the one Agda decides when it typechecks — not the identity type Id of lecture 5, which is a type

Conversion: it is an equivalence

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash M = M : A} \quad \frac{\Gamma \vdash M = N : A}{\Gamma \vdash N = M : A} \quad \frac{\Gamma \vdash M = N : A \quad \Gamma \vdash N = P : A}{\Gamma \vdash M = P : A}$$

Reflexivity, symmetry, transitivity — at a fixed type A

Reflexivity is where typing enters: only a well typed term is equal to itself

Conversion: changing the type

$$\frac{\Gamma \vdash M = N : A \quad \Gamma \vdash A = B : U \quad \Gamma \vdash B : U}{\Gamma \vdash M = N : B}$$

The companion of the typing conversion rule: an equation may be transported along an equality of types

Both rules are needed. Typing and conversion are two judgments, and each must be closed under conversion of the type

Conversion: the β rule

$$\frac{\Gamma \vdash A : U \quad \Gamma, x:A \vdash B : U \quad \Gamma, x:A \vdash M : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x:A)M) a = M[a/x] : B[a/x]}$$

The only rule that makes anything *compute*

Note where the substitution happens: in the term *and* in the type

Everything else in the list is either structural (congruence) or extensional

Conversion: congruence for Π

$$\frac{\begin{array}{l} \Gamma \vdash A : U \quad \Gamma, x:A \vdash B : U \quad \Gamma, x:A \vdash B' : U \\ \Gamma \vdash A = A' : U \quad \Gamma, x:A \vdash B = B' : U \end{array}}{\Gamma \vdash \Pi(x:A)B = \Pi(x:A')B' : U}$$

Equal domains and equal codomains give equal products

The codomains are compared in the context extended by A — *not* by A' ; the two choices are interderivable, one has to fix one

The first three premises are redundant (they follow from the last two); they are in the rule on purpose — see the slide “why so many premises”

Conversion: congruence for application

$$\frac{\Gamma \vdash A : U \quad \Gamma, x:A \vdash B : U \quad \Gamma \vdash f = f' : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a = f' a : B[a/x]}$$

$$\frac{\Gamma \vdash A : U \quad \Gamma, x:A \vdash B : U \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a = a' : A}{\Gamma \vdash f a = f a' : B[a/x]}$$

One rule for each argument, so that conversion is a congruence for application

In the second, the type of the conclusion is $B[a/x]$, with a and not a' ; by the first congruence and the type conversion rule the two are convertible anyway

Conversion: function extensionality

$$\frac{\Gamma \vdash A : U \quad \Gamma, x:A \vdash f x = g x : B \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash g : \Pi(x:A)B}{\Gamma \vdash f = g : \Pi(x:A)B}$$

Two functions of the same product type are *judgmentally* equal as soon as they agree on the *generic* argument x

This is a strong rule — it is the η principle in its extensional form. It is the rule that will require real work in the model

Compare with lecture 5: there, function extensionality was a *statement* about **Id**, which one could add as an axiom. Here it is a rule of judgmental equality

Two rules we do not need to state

There is no congruence rule for λ (usually called ξ)

$$\frac{\Gamma, x:A \vdash M = M' : B}{\Gamma \vdash \lambda(x:A)M = \lambda(x:A)M' : \Pi(x:A)B}$$

It is *derivable*: apply both sides to the generic argument x , use β twice to get $(\lambda(x:A)M)x = M = M' = (\lambda(x:A)M')x$, and conclude by function extensionality

Similarly there is no rule $f = \lambda(x:A)f\ x$: it follows from β and extensionality

A system of rules is a *choice*; what matters is which judgments are derivable. Both derivations are written out in `xirule.agda`

The conversion rules, at a glance

conv-refl, conv-sym, conv-trans	an equivalence, at a fixed type A
conv-conv	transport along $\Gamma \vdash A = B : U$
conv-beta	$(\lambda(x:A)M) a = M[a/x]$
conv-Pi	congruence for Π
conv-App-fun, conv-App-arg	congruence for application
conv-funext	$f = g$ from $f x = g x$

Seventeen rules in all, counting the congruence for application as two. Every question “is this a rule of type theory?” is now a question about this list

Why so many premises?

Several premises are *redundant*: in the application rule, $\Gamma \vdash A : U$ and $\Gamma, x:A \vdash B : U$ follow from $\Gamma \vdash f : \Pi(x:A)B$

One can leave them out. The price: one must then prove a *presupposition lemma* — if $\Gamma \vdash M : A$ then $\vdash \Gamma$ and $\Gamma \vdash A : U$ — before doing anything else, and that lemma is itself by induction on derivations

Keeping them makes every rule *self-contained*: everything one needs is a genuine *subderivation*. A function defined by induction on derivations — for instance the interpretation in a model — is then structurally recursive, and Agda accepts it directly

Cost: bigger derivations. Benefit: the proofs go through

Typing, without the redundant premises

The rules for contexts, for variables, for U and for Π lose nothing. The three others become

$$\frac{\Gamma, x:A \vdash M : B}{\Gamma \vdash \lambda(x:A)M : \Pi(x:A)B} \quad \frac{\Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[a/x]} \quad \frac{\Gamma \vdash M : A \quad \Gamma \vdash A = B : U}{\Gamma \vdash M : B}$$

A is a type because $\Gamma, x:A$ is a context, B because M has type B ; in the application rule both are read off the type of f ; and $\Gamma \vdash B : U$ is contained in $\Gamma \vdash A = B : U$

Conversion, without the redundant premises

$$\frac{\Gamma, x:A \vdash M : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x:A)M) a = M[a/x] : B[a/x]} \quad \frac{\Gamma \vdash A = A' : U \quad \Gamma, x:A \vdash B = B' : U}{\Gamma \vdash \Pi(x:A)B = \Pi(x:A')B' : U}$$

The β rule keeps the two premises which say what is substituted where; the congruence for Π keeps only the two equations

Reflexivity is unchanged, symmetry and transitivity have nothing to lose, and conv-conv drops $\Gamma \vdash B : U$, like the typing rule

The congruences, and extensionality

$$\frac{\Gamma \vdash f = f' : \Pi(x:A)B \quad \Gamma \vdash a = a' : A}{\Gamma \vdash f a = f' a' : B[a/x]}$$

$$\frac{\Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash g : \Pi(x:A)B \quad \Gamma, x:A \vdash f x = g x : B}{\Gamma \vdash f = g : \Pi(x:A)B}$$

The two congruences for application become *one*: with the redundant premises gone there is no reason to move one argument at a time

In extensionality, on the other hand, the typings of *f* and *g* are *not* redundant — nothing in $\Gamma, x:A \vdash f x = g x : B$ says that *f* and *g* are functions

What the economy costs

The two systems derive *the same judgments* — but to see it one needs the *presupposition lemma*: if $\Gamma \vdash M : A$ then $\vdash \Gamma$ and $\Gamma \vdash A : U$; if $\Gamma \vdash M = N : A$ then $\Gamma \vdash M : A$ and $\Gamma \vdash N : A$

With the rules above it is *not* provable by induction on derivations: the case of application needs the *substitution lemma*, whose case for λ needs the presupposition lemma again — each time on a derivation which is *not* a subderivation, and of unrelated height

With the premises kept, the two, and weakening with them, are *one structural induction* (selfcontained.agda). So `minimalrules.agda` proves these lemmas by *translating* between the two systems

What is *not* in the list

- no reduction relation: conversion is defined *directly*, as a judgment. Reduction is a separate notion, used to *study* the theory
- no η as a rewriting rule: extensionality instead
- no cumulativity, no subtyping, no hierarchy of universes
- no Σ , no inductive types, no **Id** — they would be more rules of exactly the same shape

Adding a type former: its formation / introduction / elimination rules, its computation rules as conversions, one congruence per constructor

Type : Type

The rule $U : U$ makes the theory *inconsistent as a logic*: there is a closed term of type $\Pi(A:U)A$, and it has *no normal form*. So the theory is not normalising, and typechecking for it is undecidable

What does hold, for exactly these rules:

- *subject reduction*: if $\Gamma \vdash M : A$ and M head-reduces to N then $\Gamma \vdash N : A$
- *injectivity of Π* : if $\Gamma \vdash \Pi(x:A)B = \Pi(x:A')B' : U$ then $\Gamma \vdash A = A' : U$ and $\Gamma, x:A \vdash B = B' : U$
- a *domain model*: types become subsets of a domain $D \cong [D \rightarrow D]$, and U an element of D

The rules, in Agda

typetype.agda: the raw syntax as **Expr** *n*, substitution, then the three judgments as *three mutually defined indexed inductive types*

- a judgment form is a *family*: HasType G M A
- an inference rule is a *constructor*, its premises are its arguments
- a derivation is an *element*; induction on derivations is *pattern matching*

Exactly the indexed inductive types of lecture 5 — the family $G \vdash X$ of deduction.agda, one step up

The file gives a full derivation of $\lambda(A:U)\lambda(x:A)x : \Pi(A:U)\Pi(x:A)A$

One rule, three notations

$$\frac{\Gamma \vdash A : U \quad \Gamma, x:A \vdash B : U}{\Gamma \vdash \Pi(x:A)B : U}$$

In Agda (`typetype.agda`)

```
ty-Pi : {n : Nat} -> {G : Ctx n} -> {A : Expr n} ->
        {B : Expr (suc n)} ->
        HasType G A U ->
        HasType (extend G A) B U ->
        HasType G (Pi A B) U
```

Same constructor, same name, in `~/DOMAIN/MIN/Syntax/Typing.agda`

What to remember

- a type theory is: a raw syntax, judgment forms, a *finite list* of rules — nothing is left to the imagination
- here three judgments, mutually defined; “ A is a type” is $A : U$
- the dependency shows up in *one* place: the type of $f a$ is $B[a/x]$ — and it forces the conversion rule
- conversion is a *judgment*, typed: β , congruences, extensionality
- a rule is a constructor of an indexed inductive family

Exercises

1. Derive $\vdash \lambda(A:U)\lambda(x:A)x : \Pi(A:U)\Pi(x:A)A$ on paper, and compare with `tyId`
2. Derive the ξ rule from β and extensionality; compare with `xirule.agda`
3. Write the rules for $A \rightarrow B$ as a *primitive* former, and derive them from the Π rules
4. Give the formation/introduction/elimination/conversion rules for $\Sigma(x:A)B$ in the same style, and add them to `typetype.agda`
5. Which rules would have to change if one replaced $U : U$ by $U_0 : U_1 : U_2 : \dots$?

References

- P. Martin-Löf. *A theory of types*. Technical report, 1971
- J.-Y. Girard. *Interprétation fonctionnelle*. . . . Thèse, 1972
- J. Reynolds. *Polymorphism is not set-theoretic*, 1984
- A. Meyer, M. Reinhold. “*Type*” is not a type, POPL 1986
- T. Coquand. *A new paradox in type theory*, 1994
- A. Hurkens. *A simplification of Girard’s paradox*, 1995
- T. Coquand. *A variation of Reynolds-Hurkens paradox*, 2024

Files: `peirce.agda`, `typetype.agda`, `xirule.agda`, `minimalrules.agda`